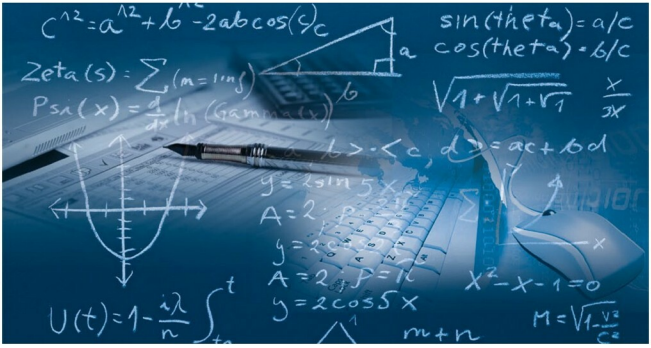
Get Analytical with R

R is a programming language and software environment for statistical computation and analysis. The author presents a way of doing statistical calculations using R in the field of analytical chemistry, particularly in titrations of various kinds



With a spreadsheet, some types of calculations are difficult to carry out. So I was on the look out for software to complement the spreadsheet. This software had to be free (as in freedom and, if possible, as in beer) and open source, powerful, lightweight, fast, used by many people, very well documented and easy to learn. All these features in one software seemed almost impossible to find. Yet, after a search, I found 'R'. Since the full code of each example presented here is too long for a magazine article, it's freely available at *https://github.com/astonfe/r*

Potentiometric titration

Titration is a common laboratory method of quantitative chemical analysis that is used to determine the unknown concentration of an identified analyte. This experiment is taken from *Reference 3* given at the end of the article, and is about the potentiometric titration curve for 2.433 meq of Cl^- with 0.1000 M $AgNO_3$. The raw data are read from an *xls* file (LibreOffice Calc) via the *gdata* package, which requires the Perl language to be installed on the computer. This is not a particular problem under GNU Linux, but Strawberry Perl (*http://strawberryperl.com*) must be installed if you're using Windows. This is not the only way, though. There are other options to read *xls* files from R. The raw data (Figure 1) can be easily plotted with the use of the following code:

```
plot(x,y,type="n")
grid(...)
points(x,y,type="o")
```

The first and the second derivatives (Figures 2 and 3) are useful to determine the inflection point. The derivatives can be calculated with two *for* loops and then plotted as previously seen for the raw data, as follows:

```
f=vector()
for(i in 2:n) {
    f[[i]]=((y[[i]]-y[[i-1]])/(x[[i]]-x[[i-1]]))
}
x1=x[2:n,drop=T]
y1=f[2:n,drop=T]
plot(x1,y1,type="n")
grid(...)
points(x1,y1,type="o")
s=vector()
for(i in 3:n) {
    s[[i]]=((f[[i]]-f[[i-1]])/(x[[i]]-x[[i-1]]))
}
x2=x[3:n,drop=T]
y2=s[3:n,drop=T]
plot(x2,y2,type="n")
```